Claude Managed Agents

A practical guide to the cookbook:
from hello world to dynamic workflows

by Daniel Ávila

October 2026

github.com/davila7/claude-managed-agents-cookbook

1. What is Claude Managed Agents?

Claude Managed Agents (CMA) is Anthropic's ready-made agent harness, running on Anthropic's infrastructure. Instead of writing your own loop (call the model, run tools, call the model again), you give Claude a **cloud sandbox** where it reads files, runs commands, browses the web, and executes code. Your application only **sends events** and **reads a stream** of what the agent is doing. Anthropic handles the rest: tools, prompt caching, context compaction, and retries.

	Messages API	Claude Managed Agents
What it is	Direct access to the model	Agent harness + managed infrastructure
Best for	Your own loops, fine-grained control	Long tasks (minutes or hours) and async work
You build	The loop, the tools, the runtime	Just your app: send events, read the stream

Four concepts explain everything:

Concept	What it is	Lifecycle
Environment	The sandbox template: a cloud container and its network	Create once, reuse
Agent	Versioned config: model, system prompt, tools	Create once; every change makes a new version
Session	An agent running in an environment for one task	One per task or conversation
Events	Messages between your app and the agent, over SSE	Many per session

CMA is in **beta**: every call carries the `managed-agents-2026-04-01` header, which the SDK and the CLI add for you. It's enabled by default for every API account.

2. A use case: when to use it

Picture a company's legal team (the repo calls it *Brightloom Inc.*). Every quarter it reviews dozens of supplier contracts: auto-renewal dates, notice periods, liability caps, change-of-control clauses. Today someone does it by hand, in a spreadsheet.

With CMA, the app uploads the contracts, creates a session, and sends one instruction: *"Review the contracts and write a summary and a risk report."* The agent reads each file, works out dates with bash, writes `summary.csv` and `report.md`, and the app downloads them. With hundreds of contracts, the agent writes a **workflow** that splits the work: one agent per contract, and another that checks each result. That's exactly the example the cookbook builds, step by step, across modules 04 to 07.

Use CMA when…	The Messages API fits better when…
The task takes minutes or hours and chains many tools	You need a quick answer in a chat
The agent needs to run code or work with files in a sandbox	You want to control every step of your own loop
The work is async: start the task, come back for the result	Your tools live only in your backend
You want persistent state (history and files) or several agents in parallel	You need Zero Data Retention or a HIPAA BAA (CMA isn't eligible)

3. Requirements

- **Python 3.10 or later.** The notebooks use `match`, which doesn't exist in 3.9. Check with `python3 --version`.
- **Visual Studio Code** with Microsoft's **Python** and **Jupyter** extensions.
- **Git**, to clone the repo.
- **An API key** from `platform.claude.com/settings/keys`, with credit on the account.
- **The `ant` CLI, version 1.40 or later**, only for the `-cli` notebooks: `brew install anthropics/tap/ant` on macOS; on Linux, download a release from GitHub.

Cost: the notebooks call the real API. Modules 01–05 cost cents each, 06 about US$0.35, and 07 about US$1. The sessions in 06 and 07 set a budget as a hard cap.

4. Setting up VS Code

1. Clone the repo and open it.

```
git clone https://github.com/davila7/claude-managed-agents-cookbook.git
cd claude-managed-agents-cookbook
code .
```

2. Create a virtual environment. From the VS Code terminal (*Terminal → New Terminal*), or with the palette (`Cmd/Ctrl+Shift+P` → *Python: Create Environment* → *Venv*):

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -U "anthropic>=1.13" python-dotenv ipykernel
```

3. Set your API key. There is a single `.env` at the repo **root**, shared by every notebook. Don't create one per folder.

```
cp .env.example .env
# edit .env:  ANTHROPIC_API_KEY=sk-ant-...
```

4. Open a notebook and pick the kernel. Open, for example, `01-basics/hello_world.ipynb`. At the top right, click **Select Kernel → Python Environments → .venv**. If the kernel isn't the venv's, you'll see `ModuleNotFoundError`.

5. Run cell by cell. `Shift+Enter` runs a cell and moves to the next. Read the text between cells: it explains each step. Skip *Run All* the first time; the point is to check the Console between one cell and the next.

Common problems

Symptom	Fix
`ModuleNotFoundError: anthropic`	The kernel isn't `.venv`: change it in Select Kernel.
`ANTHROPIC_API_KEY not found`	The `.env` is missing at the root, or the key is empty.
`APIConnectionError: Connection error.` on every call	You have an old `brotli`: `pip install -U "brotli>=1.2"` and restart the kernel.
`ant: command not found` in a CLI notebook	Install it and **restart the kernel**, or VS Code, so it picks up the new PATH.

5. How the repo is organized

Each numbered folder is a self-contained module, with a `README.md` and a notebook. Every topic comes in **two flavors**: first the Python SDK, then a **-cli** version that teaches the same thing with the `ant` CLI. The modules build on each other, so follow them in order:

```
01 basics → 02 multi-turn → 03 tools & permissions → 04 files
→ 05 session control → 06 subagents → 07 dynamic workflows
```

Item	What it's for
`NN-topic/`	Notebook with the Python SDK. This is how you integrate CMA into your application.
`NN-topic-cli/`	The same flow with `ant`: agents and environments declared in files (`agent.md`, `environment.yaml`) and applied with `ant apply`.
`sample-data/contracts/`	Six fictitious contracts used by modules 04, 06, and 07.
`claude-lock.json`	Written by `ant apply`: it records the IDs of what it created, so it doesn't duplicate them. It holds your organization's IDs, so the repo ignores it.
`outputs/`	Files the notebooks download from sessions. Also ignored by git.

SDK or CLI, which one? If you're going to write an application, start with the SDK. The CLI shines on the *control plane*: defining agents as version-controlled files, reviewing them in a pull request, and applying them from CI. You can do just the SDK modules and come back to the CLI ones later.

6. Follow along on platform.claude.com

The point of this cookbook is to **watch** each resource being created and run. Keep the Console open next to VS Code, and after each cell that creates something, refresh and find it by its ID (`env_…`, `agent_…`, `sesn_…`).

In the sidebar, under **Managed Agents**, the **Sessions** section lists every session with its status, agent, tokens, cost, and date. Open one to see:

- **Timeline:** activity over time, with one lane per thread when there are several agents.
- **Transcript:** the conversation grouped by model request, with every tool call, its input, and its result.
- **Inspector**, with five tabs: **Session** (details, cost, and budget), **Events** (each event as JSON), **Tools** (calls and failures), **Resources** (mounted files and outputs), and **Threads** (each agent, with its cost).

From the terminal, `ant beta:sessions connect <session-id>` follows a session live.

7. Module by module

No need to repeat the code: it's in each notebook, with comments. This is your guide to what to look for in each one.

Module 01 · Basics

What you learn	The full flow: environment → agent → session → event stream. The most important rule, **stream-first**: open the stream before you send the message, or you'll miss the first events.
What you run	Five cells: create the environment, create the agent, create the session, and send a message while reading the stream.
What to look at in the Console	After each cell, the new resource with its ID. The agent shows up as **version 1**. In the session, the **Transcript** shows the reply, and the **Events** tab shows everything we don't print (`span.*`, `session.usage`…).
Time and cost	1–2 minutes, cents.

Module 02 · Multi-turn

What you learn	A session keeps the conversation on the server: you send only the new message, never the history. A `chat()` helper wraps the stream-first pattern.
What you run	Three turns on the same session (Jupiter and its moons). The REPL cell is optional and interactive.
What to look at in the Console	One **single session** collecting all three turns in its Transcript.
Time and cost	2 minutes, cents.

Module 03 · Tools & Permissions

What you learn	Your code takes part in the turn. A **custom tool** (`lookup_stock`) that runs in your system, and `bash` under `always_ask`: you approve or deny each command. Reading `stop_reason`: an idle with `requires_action` is not the end.
What you run	The agent looks up three SKUs, writes a CSV (approved), and tries to delete it (denied with a message). Then the agent is updated to the `auto` policy.
What to look at in the Console	The agent moves to **version 2**. In the Transcript, the denied call; in the **Tools** tab, the calls to `lookup_stock` and to bash.
Time and cost	3 minutes, cents.

Module 04 · Files

What you learn	Upload files (Files API), mount them into the session (`/mnt/session/uploads/`), add one while the session runs, and download what the agent writes to `/mnt/session/outputs/`.
What you run	A review of five contracts, then a sixth is added. `summary.csv` and `report.md` are downloaded to `04-files/outputs/`.
What to look at in the Console	The **Resources** tab: the mounted contracts and the agent's two outputs.
Time and cost	3–4 minutes, cents.

Module 05 · Session Control

What you learn	Controls for long sessions: a **budget** (a hard cap in cents), `system.message`, **interrupt** and redirect, **reconnecting** without missing events, auditing the history, and pausing at the cap, then resuming.
What you run	One session goes through everything. The budget is lowered on purpose to hit `budget_reached`; when it's raised, the work picks up by itself.
What to look at in the Console	The **Session** tab: spend against the budget. In **Events**, the `user.interrupt` and the `system.message`.
Time and cost	2–3 minutes, cents.

Module 06 · Subagents

What you learn	The `multiagent` block: a lead agent (Opus) delegates to two specialists (Haiku), each in its own **thread**. The lead delegates **by name**: the names in the system prompt must match the agents' names.
What you run	A contract review split between the liability reviewer and the renewal reviewer; then a follow-up question to the same subagent.
What to look at in the Console	The **Timeline** with one lane per agent, and the **Threads** tab with each one's cost.
Time and cost	4–5 minutes, about US$0.35 (US$3 budget).

Module 07 · Dynamic Workflows

What you learn	The goal of the cookbook: the agent **writes a program** (a workflow) that runs many agents in phases, and the server runs it in the background. The work is done when no run is left open **and** an idle with `end_turn` arrives.
What you run	A run with phases (extract, check every extraction, redo the ones that fail): one Haiku reader per contract and inline checkers. At the end, the agent is asked to print the workflow it wrote.
What to look at in the Console	The Timeline fills with lanes: more than a dozen threads. In **Threads**, each reader's and checker's cost.
Time and cost	3–5 minutes, about US$1 (US$5 budget).

The -cli versions do the same with `ant`. The first time you run `ant apply`, it prints your organization and workspace IDs: don't share them in screenshots. Run it again and it replies *"Everything is up to date"*: it doesn't duplicate resources.

8. Golden rules

Rule	Why
Stream-first	A stream only delivers events emitted after it opens.
An idle isn't the end	Check `stop_reason`: `end_turn`, `requires_action`, `budget_reached`… With workflows, also wait until every run has ended.
Answer each call as it arrives	With several agents, the session can stay `running` while one of them waits on you.
Create once, reuse IDs	Environments and agents are configuration; sessions are runs.
Set a budget	Especially with subagents and workflows: one cap covers every agent.
Download before deleting	Deleting a session also deletes the files it produced.

9. What's next?

- **Scale up the data:** put hundreds of contracts in `sample-data/` and watch the module 07 run split the work.
- **Define outcomes:** a grader checks the deliverable against a rubric and asks for revisions until it passes.
- **Webhooks and scheduled deployments:** get notified when a long session finishes, or run a review like this one on a schedule.

Official docs: `platform.claude.com/docs/en/managed-agents/overview` · Repo: `github.com/davila7/claude-managed-agents-cookbook`